

Relatório do Trabalho T1

Segurança em Computação – 2025/2

Infraestrutura de Certificação Digital: PKI Automatizada (Python) e Manual (OpenSSL)

Informações do Grupo

- **Disciplina:** Segurança em Computação 2025/2
- **Integrantes:**
 - Nome: Gustavo Sarti
 - Nome: José Vitor

1. Arquitetura do Ambiente

A arquitetura implementada simula uma Infraestrutura de Chaves Públicas (PKI) corporativa de 3 níveis, garantindo uma cadeia de confiança robusta para ambientes internos sem depender de validadores externos.

- **Cenário 1: PKI Automatizada (Python)**
Neste cenário, utilizamos um script desenvolvido em Python (com a biblioteca cryptography) que atua como uma autoridade certificadora automatizada. O script gera as chaves, define as extensões X.509 e realiza as assinaturas digitais em memória, exportando os certificados finais prontos para uso no Nginx.
- **Cenário 2: PKI Manual (OpenSSL)**
Neste cenário, assumimos o papel de administradores da CA, executando cada etapa manualmente via terminal. Criamos as requisições de assinatura (CSR), configuramos arquivos de extensão (.cnf) para garantir a conformidade (ex: SAN para localhost) e assinamos os certificados explicitamente.

Diagrama da Cadeia de Confiança (Válido para ambos):

[CA Raiz (Autoassinada)] → [CA Intermediária] → [Servidor Localhost]

2. Tarefa 1 – HTTPS com PKI Automatizada (Python)

2.1. Preparação do Ambiente

- **Sistema operacional:** Windows (com Docker Desktop e Python instalado)
- **Ferramentas utilizadas:** VS Code, Docker, Python 3.x, Biblioteca cryptography.
- **Versão do Docker / Nginx:** Latest
- **Configuração do Servidor Web:**
O servidor Nginx foi configurado através do docker-compose.yml para responder na porta 443 (HTTPS), montando um volume que aponta para a pasta onde o script Python deposita os certificados gerados.

Trecho do nginx.conf:

```
server {  
    listen 443 ssl;  
    server_name localhost;  
    ssl_certificate /etc/nginx/certs/fullchain.crt;  
    ssl_certificate_key /etc/nginx/certs/server.key;  
    ...  
}
```

2.2. Automação com Python

- **Script Utilizado:** pki_script.py
- Explicação:
A validação do domínio é feita internamente pela PKI privada. O script cria uma CA Raiz (RSA 4096 bits) autoassinada. Em seguida, cria uma CA Intermediária assinada pela Raiz. Por fim, gera a chave do servidor e emite um certificado para CN=localhost, garantindo a inclusão da extensão Subject Alternative Name (SAN) necessária para navegadores modernos (Chrome/Edge).

2.3. Emissão do Certificado

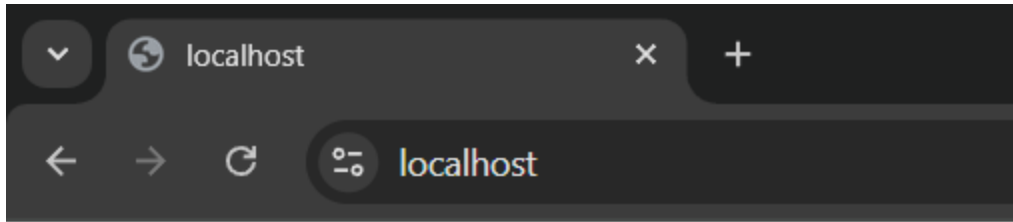
- **Caminho do certificado gerado:** T1_Python/certs/fullchain.crt
- Processo de Validação:
O script executa a assinatura digital usando as classes CertificateBuilder da biblioteca cryptography. Ele gera os arquivos .key (privados) e .crt (públicos) e concatena o certificado do servidor com o da intermediária para formar o fullchain, essencial para que o cliente valide a cadeia completa.

2.4. Configuração HTTPS no Nginx

- Descrição:
No arquivo docker-compose.yml, as diretivas de volume mapeiam os arquivos gerados pelo script para dentro do container. Isso habilita o TLS 1.2/1.3 no servidor, permitindo conexões seguras assim que o container sobe.

2.5. Resultados e Validação

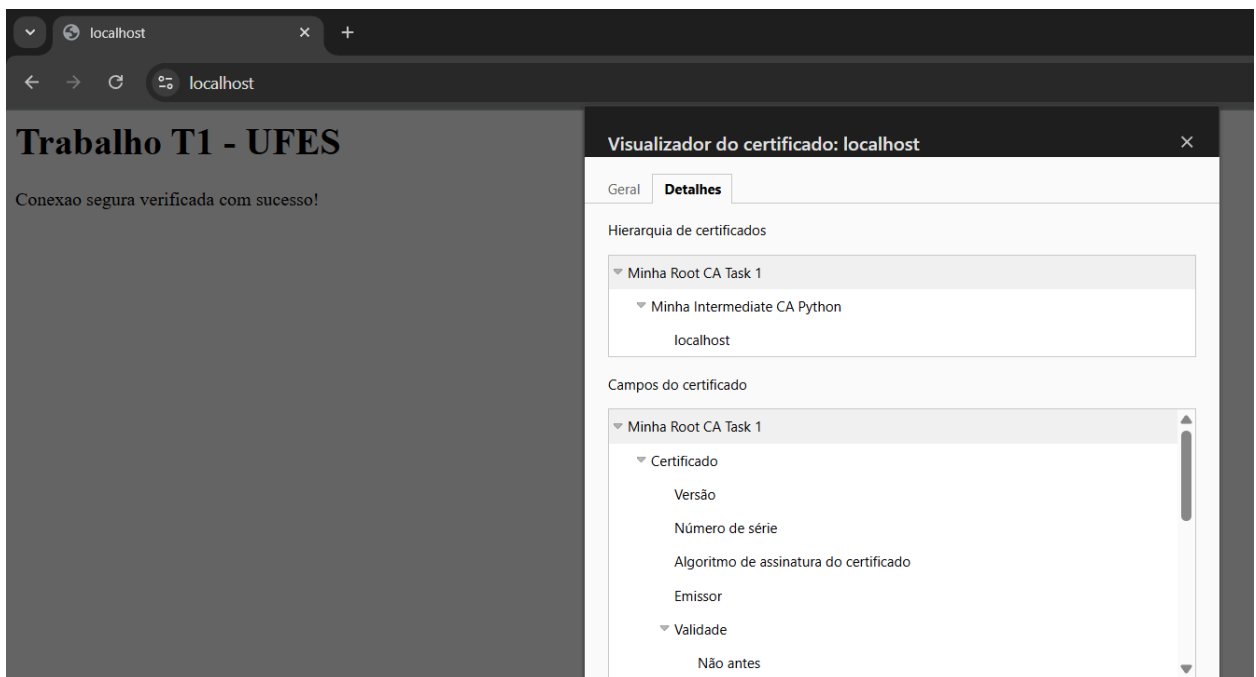
- **URL de acesso:** https://localhost
- **Screenshot da página HTTPS:**



Trabalho T1 - UFES

Conexao segura verificada com sucesso!

-
- **Resultado da verificação:** O script executou com sucesso, gerando a árvore de certificados e exibindo logs de confirmação no terminal.
- **Screenshot do certificado no navegador:**



- *Minha Root CA Python -> Intermediária -> localhost)*

3. Tarefa 2 – HTTPS com PKI Própria (OpenSSL)

3.1. Criação da CA Raiz

- **Papel e Processo:**
A CA Raiz é a âncora de confiança. Foi criada uma chave RSA 4096 bits e um certificado autoassinado (`openssl req -x509`) com validade de 10 anos. Ela é a única entidade em que o sistema operacional precisa confiar explicitamente.

3.2. Criação da CA Intermediária

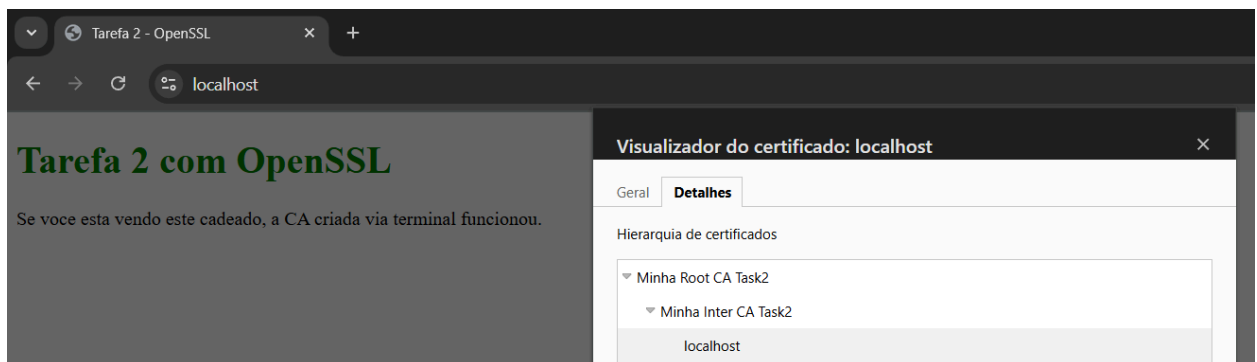
- **Processo e Segurança:**
A CA Intermediária foi criada gerando uma CSR (`openssl req -new`) que foi assinada pela chave da Raiz. O uso da intermediária permite manter a chave da Raiz offline (em um cofre, por exemplo), aumentando a segurança da infraestrutura, pois em caso de comprometimento, apenas a intermediária precisa ser revogada.

3.3. Emissão do Certificado do Servidor

- **Caminho do fullchain.crt:** T2_OpenSSL/certs/fullchain.crt
- **Processo:**
Foi gerada uma CSR para o servidor (CN=localhost). A assinatura foi feita pela CA Intermediária utilizando o arquivo de configuração `server_ext.cnf` para injetar a extensão `subjectAltName=DNS:localhost`. O fullchain foi montado concatenando `server.crt` e `inter_ca.crt`.

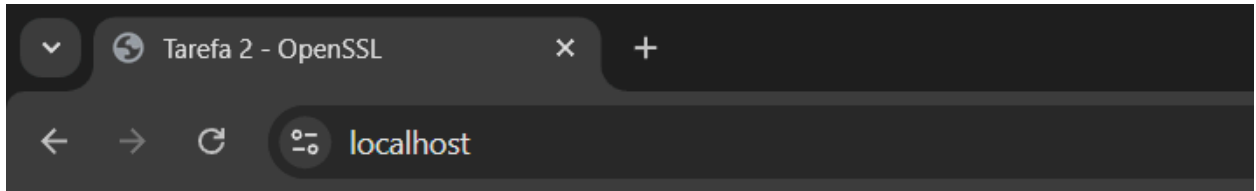
3.4. Importação da CA Raiz no Navegador

- **Procedimento:**
Utilizou-se o `certmgr.msc` no Windows. O arquivo `root_ca.crt` foi importado para o repositório de "Autoridades de Certificação Raiz Confiáveis".
- **Resultado:** O navegador passou a confiar na CA? **Sim.**
- **Justificativa:** Ao adicionar a Raiz ao repositório de confiança do Windows, o sistema operacional consegue validar matematicamente toda a cadeia de assinaturas (Server -> Intermediária -> Raiz), fechando o cadeado HTTPS.
- **Screenshot:**



3.5. Validação da Cadeia

- **Comando de verificação:** `openssl verify -CAfile root_ca.crt -untrusted inter_ca.crt server.crt`
- **Resultado:** server.crt: OK
- **Screenshot do navegador:**



Tarefa 2 com OpenSSL

Se voce esta vendo este cadeado, a CA criada via terminal funcionou.

•

4. Comparação entre os Dois Cenários

- Certificados Públicos vs. Privados:
Certificados públicos (como Let's Encrypt) são confiados nativamente por todos os dispositivos na internet, ideais para sites públicos. Certificados privados (como os deste trabalho) exigem instalação manual da Raiz nos clientes, sendo restritos a ambientes internos ou de teste.
- Cenários de Uso:
A abordagem com Python (Automação) é ideal para DevOps e esteiras de CI/CD onde certificados efêmeros são necessários para testes automatizados. A abordagem com OpenSSL (Manual) é crucial para entender o funcionamento baixo nível da criptografia e para a cerimônia de criação de CAs raízes reais, onde o controle humano é rigoroso.
- Necessidade da Importação da Raiz:
No segundo cenário, a importação é obrigatória porque a nossa "Minha Root CA" não é uma autoridade global (como DigiCert ou GlobalSign). O navegador não a conhece de fábrica, tratando-a como desconhecida até que o usuário a adicione manualmente como confiável.

5. Vídeo explicativo:

<https://youtu.be/AwbZQDDrTZM>

5. Conclusões

O trabalho permitiu consolidar o conhecimento sobre a hierarquia de confiança X.509. Foi possível evidenciar que a segurança do HTTPS depende estritamente da integridade da chave privada da CA Raiz. As lições aprendidas incluem a manipulação de extensões SAN (críticas para navegadores modernos), a diferença entre certificados autoassinados e cadeias completas, e a prática de gerenciamento de certificados no sistema operacional.

Checklist Final

Item	Status
Servidor Nginx funcional (Docker)	✓
Tarefa 1 (Python) funcional	✓
Tarefa 2 (OpenSSL) funcional	✓
Importação da CA raiz no navegador	✓
Cadeia de certificação validada com sucesso	✓
Relatório completo e entregue	✓
Apresentação prática (vídeo)	✓